



테니스 경기 규칙 및 스코어링 로직 가이드

Team : 내 컴퓨터에선 됨

문규리 신창만 오수빈

01. 경기 기본 구조

매치(Match)부터 포인트(Point)까지의 계층적 구조 이해

3세트 매치 전체 흐름

1세트 시작

0-0에서 시작,
6게임 선취 목표

2세트 진행

세트 스코어 1:0
또는 0:1 상태

승부 결정

세트 스코어 2:0 시
경기 즉시 종료

파이널 세트

1:1 상황 발생 시
3세트에서 최종 승부

* 3세트 2선승제(Best of 3)를 기본으로 하며, 각 세트는 독립적인 게임 스코어를 가집니다.

* 5세트인 경우 3선승제(Best of 5)를 기본으로 하며, 각 세트는 독립적인 게임 스코어를 가집니다.

게임 내 포인트 시스템

0 포인트: Love

경기의 시작점입니다. 프랑스어 'L'oeuf'(달걀)에서 유래되었습니다.

1 포인트: 15

첫 번째 득점 시 15점을 획득합니다.

2 포인트: 30

두 번째 득점 시 30점을 획득합니다.

3 포인트: 40

세 번째 득점 시 40점을 획득하며, 다음 득점 시 게임을 가져갑니다.



듀스 (Deuce) 및 어드밴티지 로직



듀스 발생

두 선수가 모두 3포인트를 획득하여
40-40이 된 상태입니다. 코딩 시 스코어
평형 상태를 체크해야 합니다.



어드밴티지

듀스 후 먼저 1점을 딴 선수에게
주어집니다. (Ad-In: 서버 유리, Ad-Out:
리시버 유리)



게임 종료 조건

어드밴티지 상태에서 해당 선수가 다시
득점하면 게임 승리, 상대가 득점하면 다시
듀스로 복귀합니다.

코딩을 위한 핵심 조건부 로직

IF

Conditional Branches

포인트 획득 시 체크 리스트

1. 현재 스코어가 40-40(듀스)인가?
2. 포인트 획득 후 상대와의 차이가 2점 이상인가?
3. 게임 스코어가 6-6이 되어 타이브레이크 모드인가?
4. 세트 스코어가 2-0 또는 2-1이 되어 Match Over인가?
(5세트인 경우 3-0 또는 3-1 또는 3-2가 되어 Match Over인가?)

타이브레이크 (Tie-break) 규칙

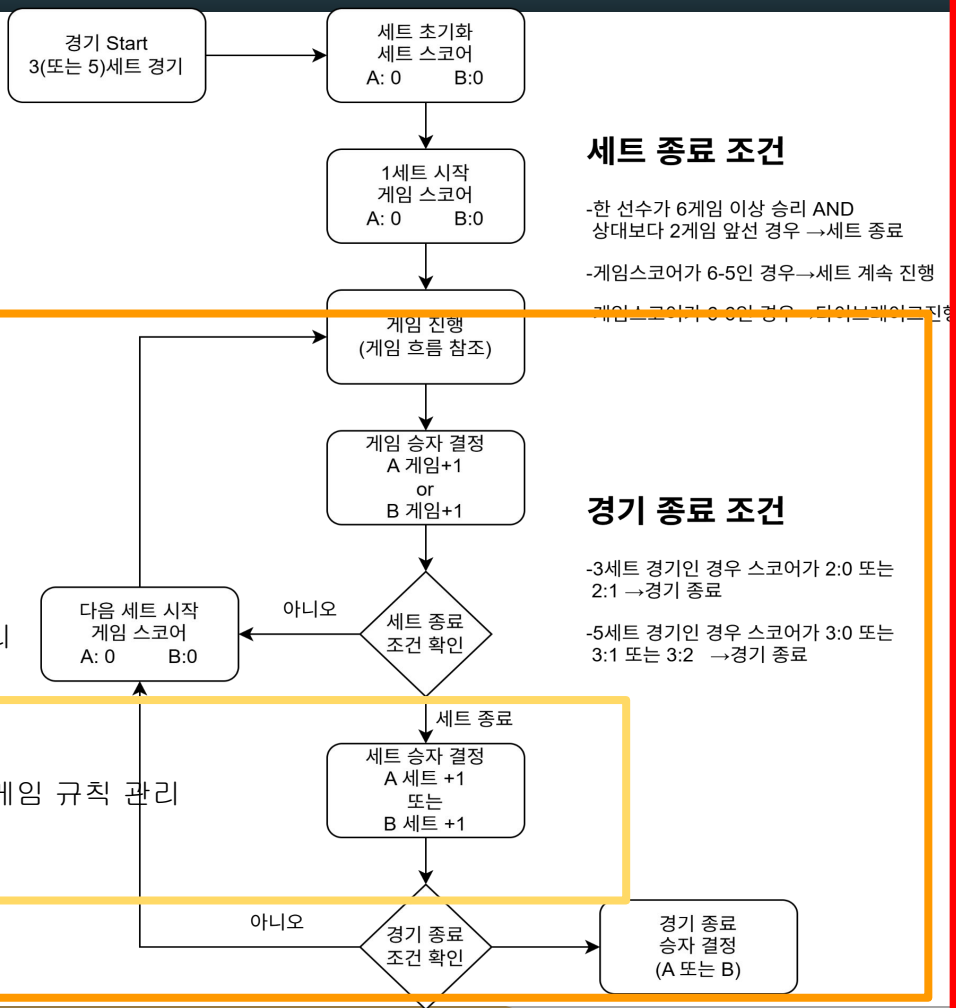
- ✔ **숫자 스코어 사용:** 15, 30 방식 대신 1, 2, 3... 단순 정수 스코어를 사용합니다.
- ✔ **7점 선취:** 먼저 7포인트를 획득하는 선수가 승리합니다. (단, 2점 차 필요)
- ✔ **서브 교체:** 첫 서버는 1회 서브 후, 이후부터는 각 선수당 2회씩 서브권을 가집니다.
- ✔ **무한 연장:** 7-6 이나 6-7 상황에서는 한 선수가 2점 차를 벌릴 때까지 계속 진행됩니다.

다이어그램(플로우 차트)

- Match = 경기 전체 관리

- SetScore = 세트 관리

- GameScore = 게임 규칙 관리

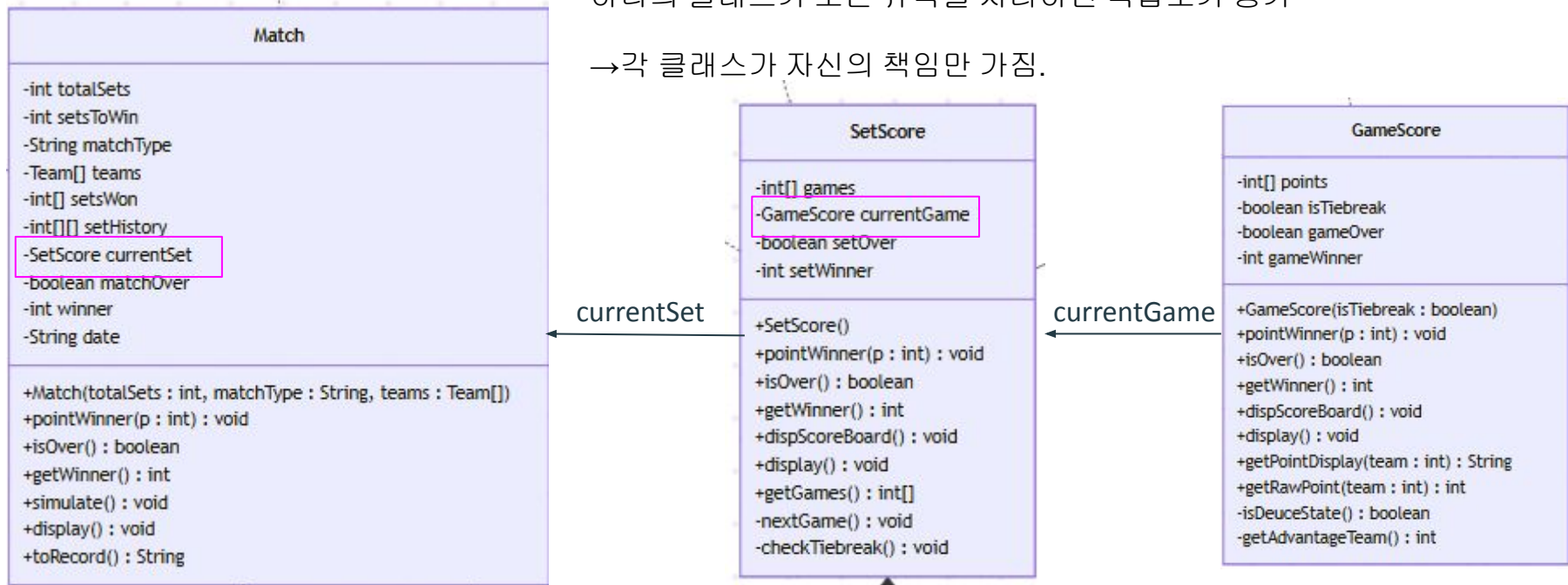


다이어그램(플로우 차트) UML에서도 3개의 계층으로 나누어 설계.

- Match는 경기 전체를,
- SetScore는 세트를,
- GameScore는 게임 규칙을 담당.

하나의 클래스가 모든 규칙을 처리하면 복잡도가 증가

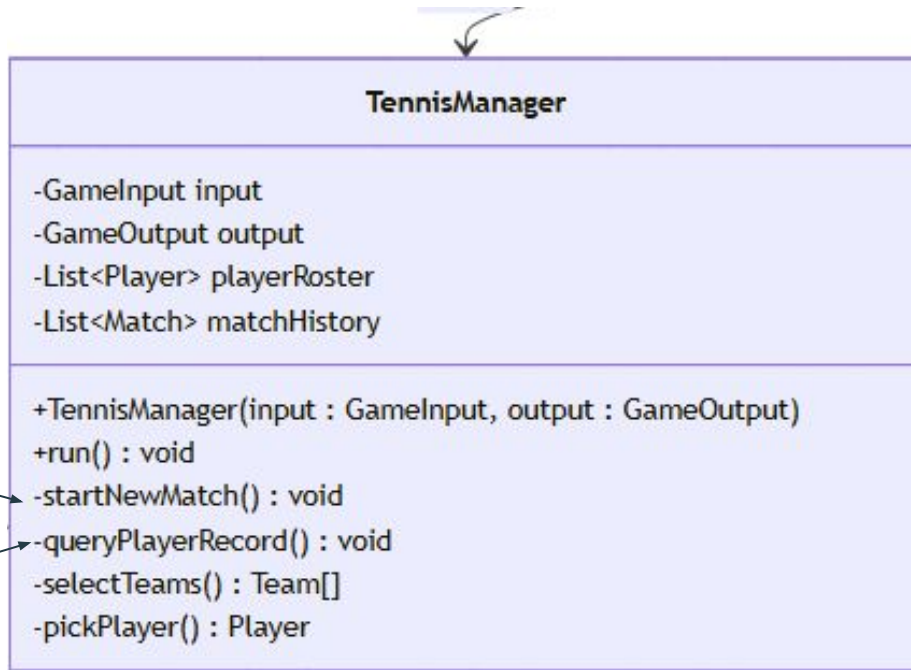
→ 각 클래스가 자신의 책임만 가짐.



다이어그램(플로우 차트)

프로그램의 전체 흐름을 제어하는 컨트롤러

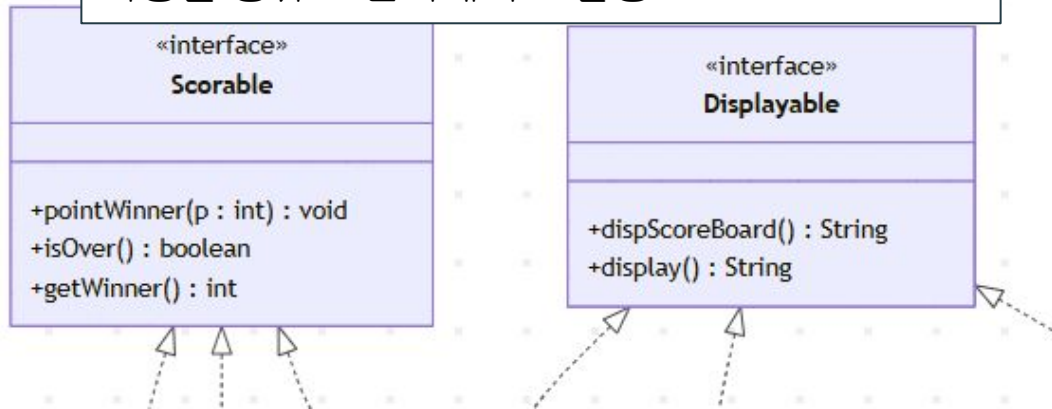
- UM메뉴 선택
- run()
- 경기 생성
- startNewMatch()
- 기록 조회
- queryPlayerRecord()



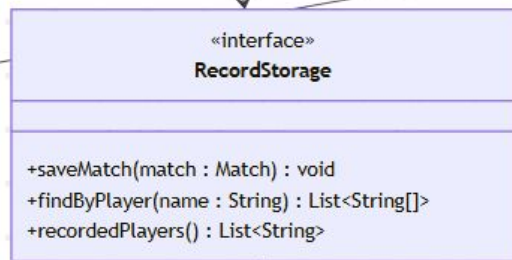
다이아그램(플로우 차트)

- 인터페이스 분리
- 공통 기능을 인터페이스로 추상화하여 일관성을 확보하고, 각 클래스에서 목적에 맞게 구현하도록 설계

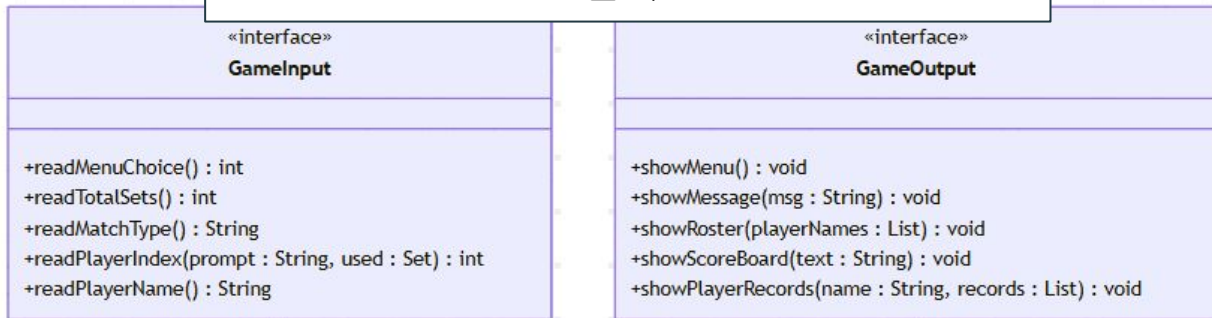
Match, SetScore, GameScore은 점수 기능, 출력 기능을 공유 → 인터페이스 활용



경기 기록 저장을 인터페이스 기반으로 설계



사용자 입력과 출력을 인터페이스를 통해 분리.

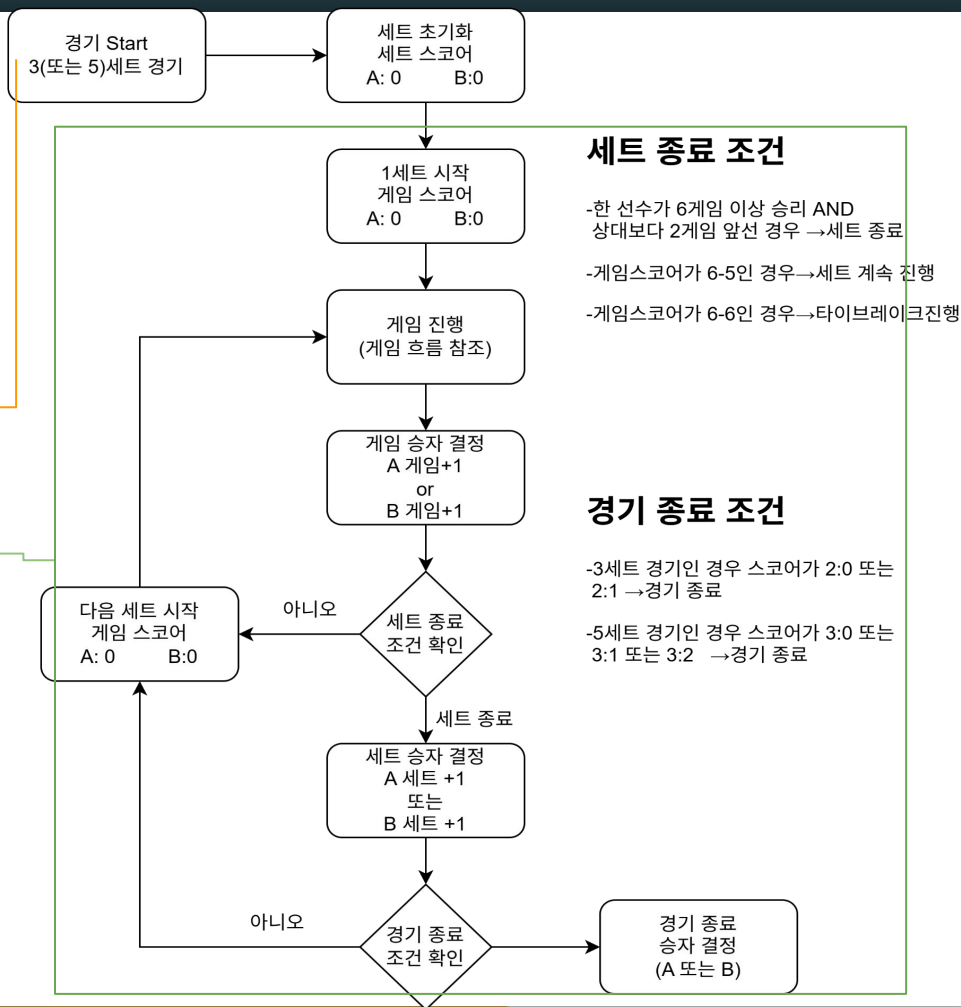


다이아그램(플로우 차트)

TennisManager 내의 startNewMatch()가 경기 시작

```
private void startNewMatch() {  
    int totalSets = input.readTotalSets();  
    String matchType = input.readMatchType();  
    Team[] teams = selectTeams(matchType);  
  
    Match match = new Match(totalSets, matchType, teams);  
    output.showMessage("\n경기를 시작합니다...");  
    match.simulate(input, output);  
    output.showScoreBoard(match.dispScoreBoard());  
  
    matchHistory.add(match);  
    storage.saveMatch(match);  
}
```

경기 종료 후 기록 저장.



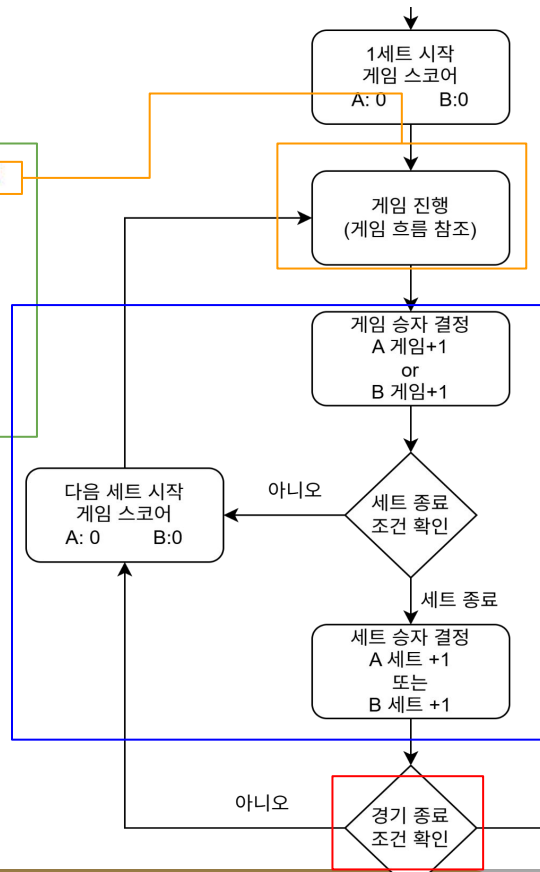
다이아그램(플로우 차트)

Match 클래스 내의 simulate()가 경기 진행

```
public void simulate(GameInput input, GameOutput output) {  
    boolean stepMode = true;  
    while (!matchOver) {  
        int team = Math.random() < 0.5 ? Team.TEAM_A : Team.TEAM_B;  
        pointWinner(team);  
        output.showScoreBoard(disScoreBoard());  
        if (stepMode && !matchOver) {  
            stepMode = input.waitForContinue();  
        }  
    }  
}
```

게임, 세트 등 결과
출력

while문 사용하여 경기가 종료되기 전까지
게임진행→출력→종료조건 확인을 반복.



세트 종료 조건

- 한 선수가 6게임 이상 승리 AND 상대보다 2게임 앞선 경우 → 세트 종료
- 게임스코어가 6-5인 경우 → 세트 계속 진행
- 게임스코어가 6-6인 경우 → 타이브레이크 진행

경기 종료 조건

- 3세트 경기인 경우 스코어가 2:0 또는 2:1 → 경기 종료
- 5세트 경기인 경우 스코어가 3:0 또는 3:1 또는 3:2 → 경기 종료

다이어그램(플로우 차트)

TennisManager 내의 queryPlayerRecord()가
result.txt로부터 저장된 경기 기록 호출

입력 받은 선수의 이름 또는
번호로 기록 서치 가능.

```
private void queryPlayerRecord() {  
    // TODO : 선수 이름 목록 보여주기  
    RecordStorage str = storage;  
    List<String> recordedPlayersList = str.recordedPlayers();  
    Iterator<String> isPL = recordedPlayersList.iterator();  
    int n = 1;  
    while (isPL.hasNext()) {  
        System.out.print(n + ".\s" + isPL.next() + "\t\t\t");  
        if(n%3==0) System.out.println();  
        n++;  
    }  
    System.out.println();  
}
```

```
String name = input.readPlayerName();  
int i = -1;  
try {  
    i = Integer.parseInt(name);  
} catch (NumberFormatException e) {  
}
```

입력받은 값이 "1"과 같은
문자형으로 된 정수일 때
try-catch로 해당 번호의 선수
찾기.

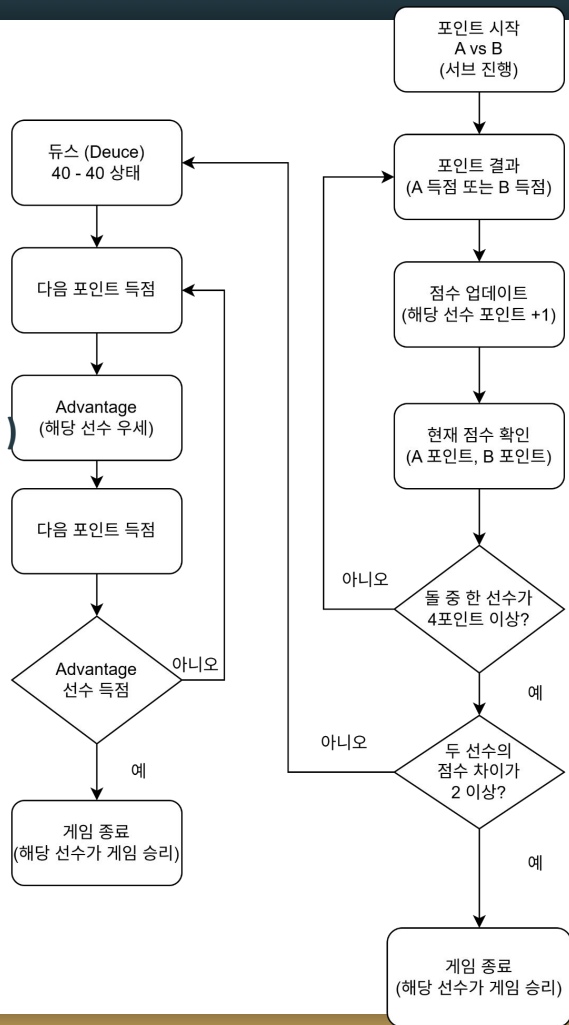
```
name = (i != -1 && i<recordedPlayersList.size()) ? recordedPlayersList.get(i-1) : name;  
List<String[]> records = storage.findByPlayer(name);  
output.showPlayerRecords(name, records);  
}
```

//경기에 참여했던 선수들만 반환.

```
public List<String> recordedPlayers() {  
    List<String> playerList = new ArrayList<String>();  
    try (BufferedReader br = new BufferedReader(new FileReader(RESULTS_FILE))) {  
        String line;  
        Map<String, String> block = new LinkedHashMap<>();  
        while ((line = br.readLine()) != null) {  
            if (line.equals("---")) {  
                String[] team1 = block.getDefault("TEAM1", "").split("\\s\\s");  
                String[] team2 = block.getDefault("TEAM2", "").split("\\s\\s");  
                int index = 0;  
                while(index<team1.length) {  
                    playerList.add(team1[index]);  
                    playerList.add(team2[index]);  
                    index++;  
                }  
                block.clear();  
            } else if (line.contains("=")) {  
                String[] kv = line.split("=", 2);  
                block.put(kv[0], kv[1]);  
            }  
        }  
        playerList = playerList.stream().distinct().toList();  
    } catch (IOException ignored) {  
        // 파일 없으면 빈 목록 반환  
    }  
    return playerList;  
}
```

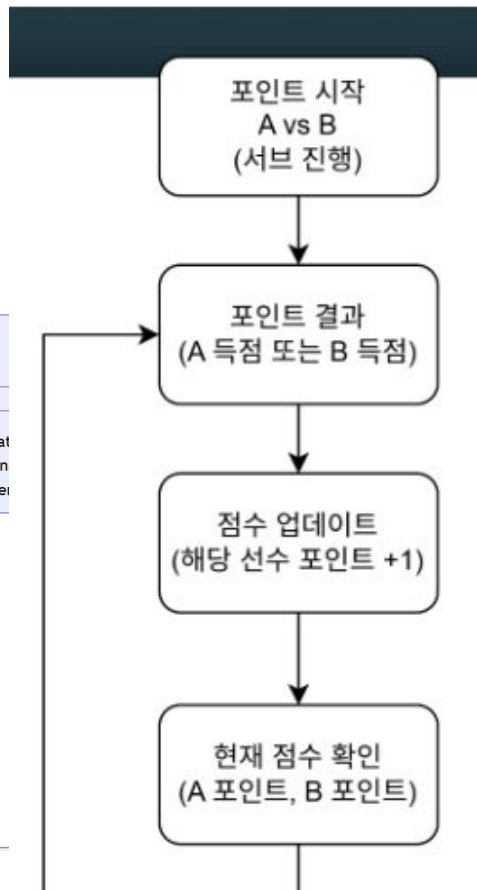
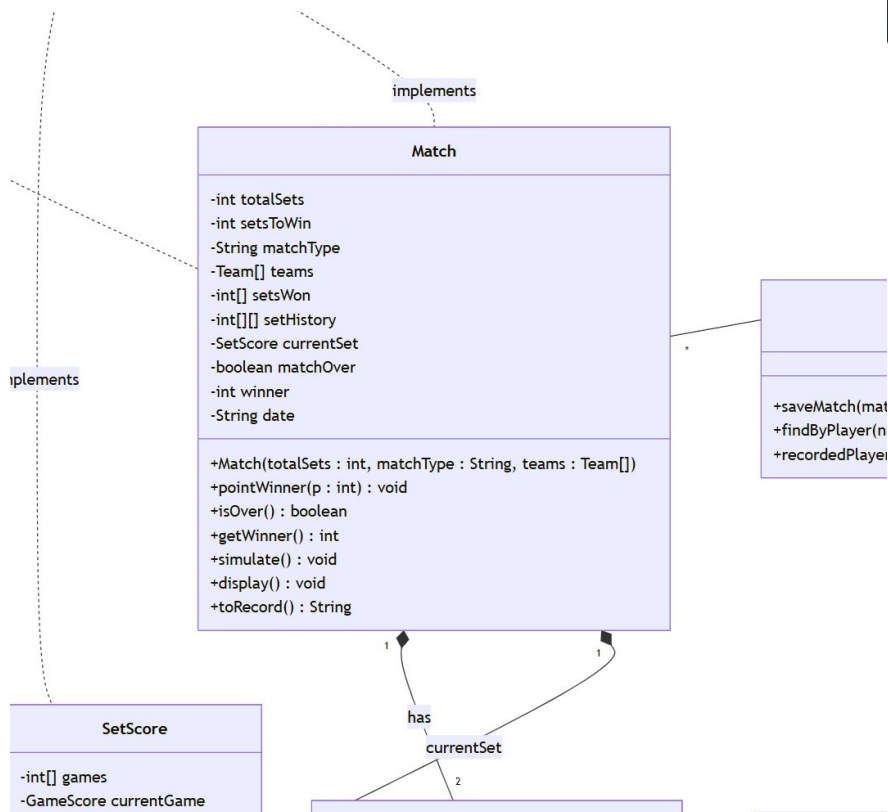
경기에 참여했던 선수들만 반환
Stream 사용. (distinct())

게임 흐름 (한 게임 내에서 포인트 진행)



포인트 점수 체계	
0	0 (Love)
1	15
2	30
3	40
4 이상	Advantage 또는 게임

※ 실제 표시: 15, 30, 40, Advantage, Deuce



pointWinner():
득점 처리 핵심 메서드

포인트(Point)

↓
게임(Game)

↓
세트(Set)

↓
매치(Match)

테니스의 전체 승리
구조를 관리하는 메서드

1
currentGame

GameScore

-int[] points
-boolean isTiebreak
-boolean gameOver
-int gameWinner

+GameScore(isTiebreak : boolean)
+pointWinner(p : int) : void
+isOver() : boolean
+getWinner() : int
+dispScoreBoard() : void
+display() : void
+getPointDisplay(team : int) : String
+getRawPoint(team : int) : int
-isDeuceState() : boolean
-getAdvantageTeam() : int

아니오

현재 점수 확인
(A 포인트, B 포인트)

둘 중 한 선수가
4포인트 이상?

예

dispScoreBoard() :

점수 표시를 담당하는 핵심
메서드

현재 게임 상태를 판별하여
화면에 출력

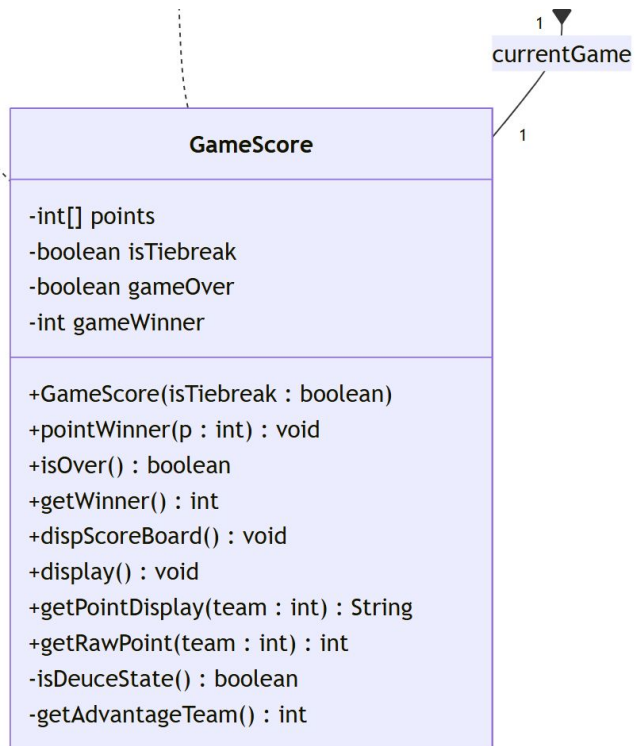
getPointDisplay()

일반 점수 변환

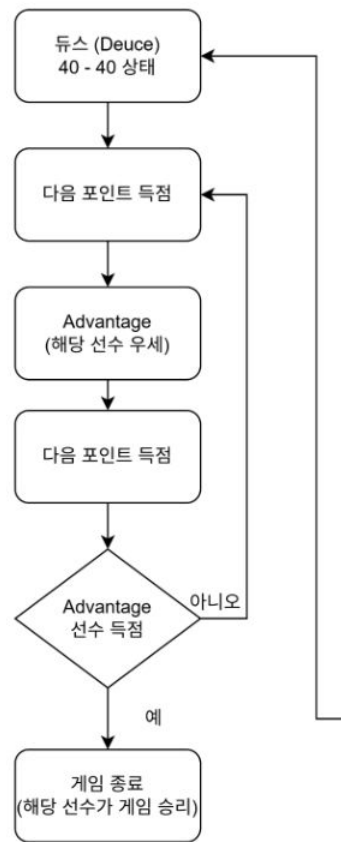
0 → 15 → 30 → 40 변환

```
@Override
public String dispScoreBoard() {
    StringBuilder sb = new StringBuilder("[현재 게임 포인트] ");
    if (isTiebreak) {
        sb.append(String.format("%d - %d (Tie-break)%n", points[Team.TEAM_A], points[Team.TEAM_B]));
    } else if (isDeuceState()) {
        sb.append("Deuce\n");
    } else if (getAdvantageTeam() != 0) {
        sb.append(getAdvantageTeam() == Team.TEAM_A ? "Adv - 40\n" : "40 - Adv\n");
    } else {
        sb.append(String.format("%s - %s\n", getPointDisplay(Team.TEAM_A), getPointDisplay(Team.TEAM_B)));
    }
    return sb.toString();
}
```

```
@Override
public String display() {
    return dispScoreBoard();
}
```



!



<테니스 특수 규칙 처리>

isDeuceState()
40:40 상태 판별

getAdvantageTeam()
Advantage 판별

isTiebreak
Tie-break 여부 판별

SetScore

-int[] games
 -GameScore currentGame
 -boolean setOver
 -int setWinner
 checkSetWin() : void
 +SetScore()
 +pointWinner(p : int) : void
 +isOver() : boolean
 +getWinner() : int
 +dispScoreBoard() : void
 +display() : void
 +getGames() : int[]
 -nextGame() : void
 -checkTiebreak() : void

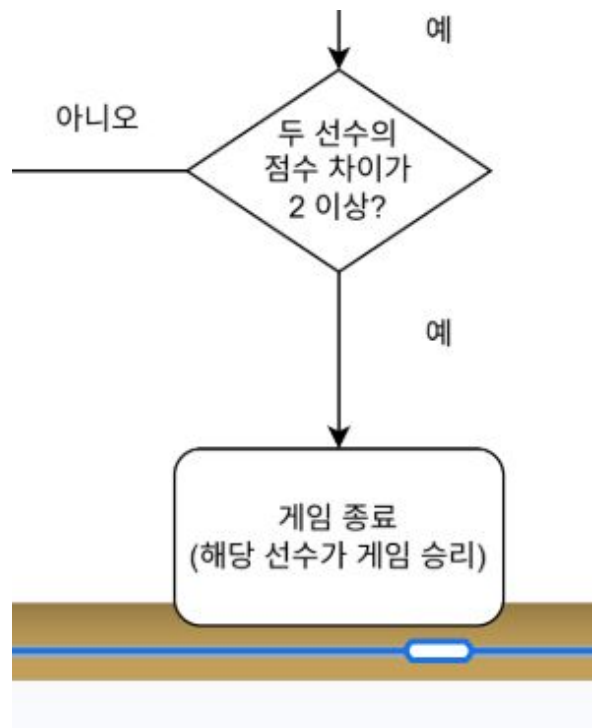
1
 currentGame

```
private void checkSetWin() {
    int g0 = games[Team.TEAM_A], g1 = games[Team.TEAM_B];
    if ((g0 >= 6 || g1 >= 6) && Math.abs(g0 - g1) >= 2) {
        setOver = true;
        setWinner = g0 > g1 ? Team.TEAM_A : Team.TEAM_B;
    } else if (g0 == 7 && g1 == 6) {
        setOver = true;
        setWinner = Team.TEAM_A;
    } else if (g1 == 7 && g0 == 6) {
        setOver = true;
        setWinner = Team.TEAM_B;
    }
}
```

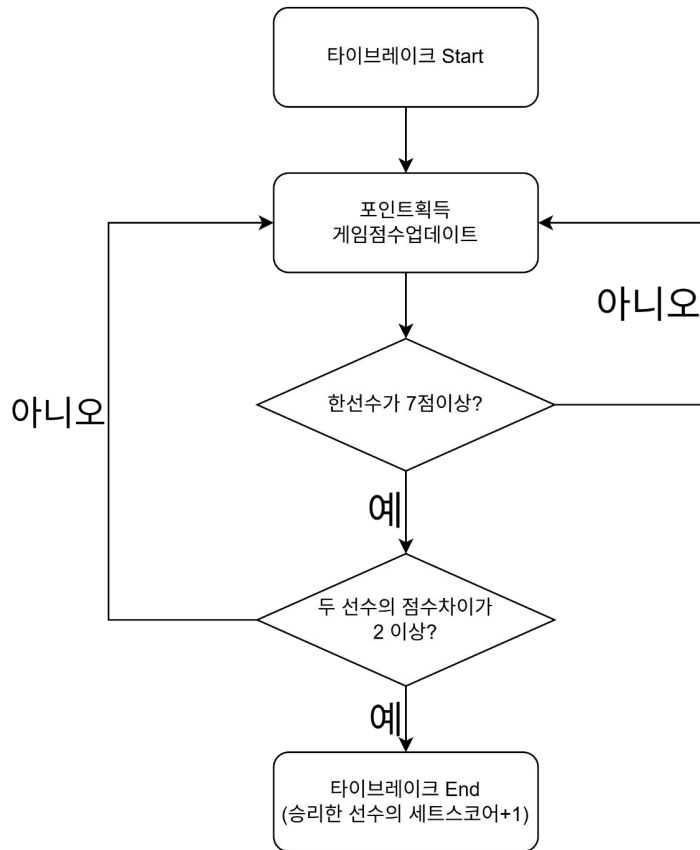
checkSetWin()

세트 종료 여부를 판단하는 핵심
 메서드

1. 6게임 이상 + 2게임 차이
 → 세트 승리
2. 6:6 이후 타이브레이크
 → 7:6 승리 가능
3. 승리 팀 저장



타이브레이크 흐름(게임스코어 6-6인경우)



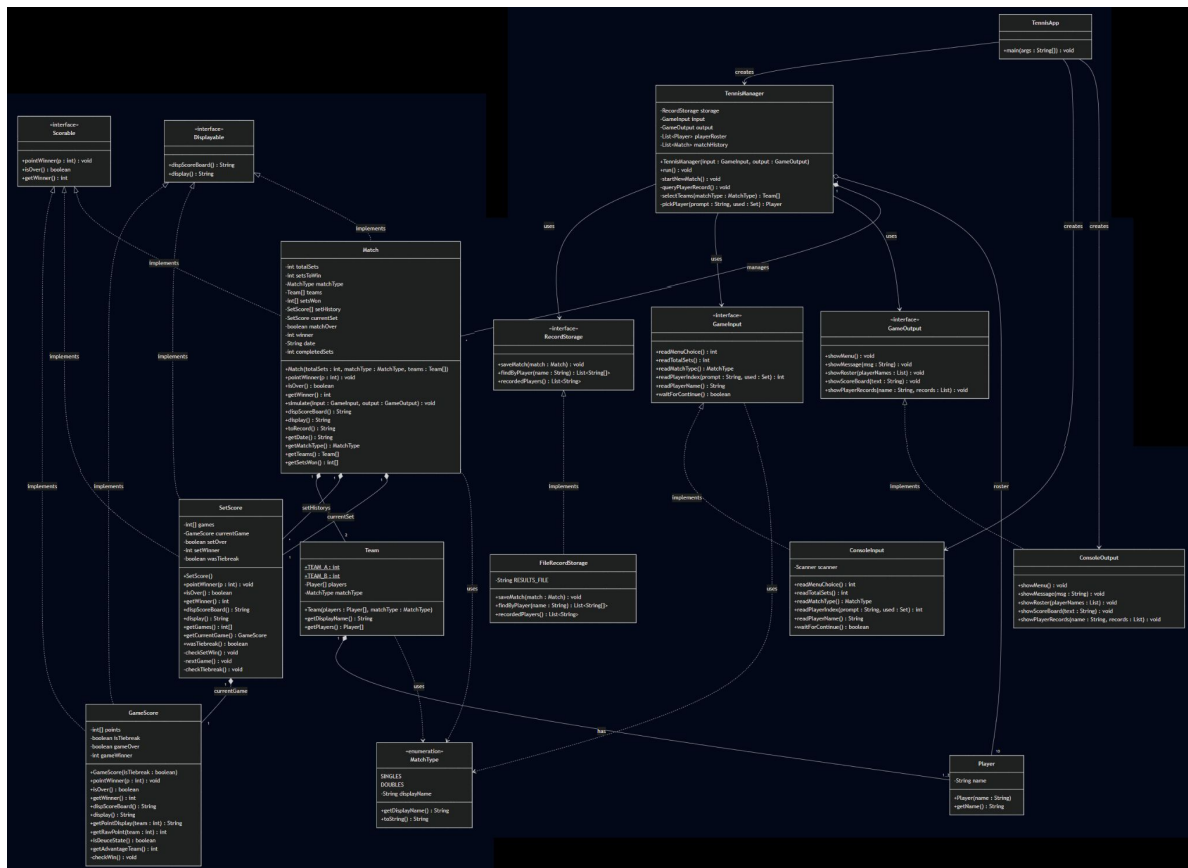
타이브레이크 규칙

점수는 0,1,2,3,... 으로 진행

먼저 7점을 획득하고, 상대보다 2점이상 앞서야 승리

(예: 7-0, 7-1... 7-5 , 8-6, 9-7 10-8...)

UML



LINK

화면 구성

===== 테니스 게임 =====

1. 게임 플레이

2. 게임 기록 조회

3. 종료

선택:

===== 테니스 게임 =====

1. 게임 플레이

2. 게임 기록 조회

3. 종료

선택: 1

경기 세트 수를 입력하세요 (3 또는 5): 3

경기 유형을 선택하세요 (1: 단식 / 2: 복식): 2

[선수 목록]

1. 양인석

2. 조지훈

3. 이지훈

4. 안정빈

5. 장미성

6. 류호훈

7. 이시연

8. 신창만

9. 오수빈

10. 문규리

[팀 A 선수 선택]

팀 A 선수 1 선택 (번호 입력):

```

[팀 A 선수 선택]
팀 A 선수 1 선택 (번호 입력): 3
팀 A 선수 2 선택 (번호 입력): 5
[팀 B 선수 선택]
팀 B 선수 1 선택 (번호 입력): 2
팀 B 선수 2 선택 (번호 입력): 7

```

경기를 시작합니다...

```

=====
현재 스코어보드
경기 방식: 3세트 복식

```

팀	세트	게임	현재	게임	포인트
이지훈 / 장미성	0	0	15		
조지훈 / 이시연	0	0	0		

```

=====
Press Enter to continue...

```

```

=====
Press Enter to continue...

```

```

=====
현재 스코어보드
경기 방식: 3세트 복식

```

팀	세트	게임	현재	게임	포인트
이지훈 / 장미성	0	2	30		
조지훈 / 이시연	1	5	40		

```

=====
Press Enter to continue...

```

```

=====
경기 종료
경기 방식: 3세트 복식

```

팀 1	승수	1세트	2세트
이지훈 / 장미성	0	1	2
조지훈 / 이시연	2	6	6

최종 승자: 조지훈 / 이시연 (2 - 0)

```

=====
경기 종료
경기 방식: 3세트 복식

```

팀 1	승수	1세트	2세트
이지훈 / 장미성	0	1	2
조지훈 / 이시연	2	6	6

최종 승자: 조지훈 / 이시연 (2 - 0)

===== 테니스 게임 =====

1. 게임 플레이

2. 게임 기록 조회

3. 종료

선택:

1~3 중에서 선택하세요:

1~3 중에서 선택하세요: 3
프로그램을 종료합니다.

1~3 중에서 선택하세요: 2

1. 양인석

2. 조지훈

3. 신창만

4. 이지훈

5. 장미성

6. 이시연

조회할 선수 이름을 입력하세요: 양인석

[양인석 게임 기록]

날짜	방식	상대	스코어	결과
2026-06-11	3세트 단식	조지훈	6-3, 5-7, 6-2	승리
2026-06-11	3세트 단식	신창만	3-6, 6-4, 3-6	승리
2026-06-11	3세트 단식	조지훈	7-6(2), 3-6, 7-6(3)	승리

총 3경기 | 3승 0패

느낀 점

문규리: 유지보수만 해오다가 처음으로 클래스 설계부터 시작하게 되었는데, 코드를 작성하면서 각 메소드들의 존재 이유를 이해하는 데에 도움이 되었습니다. 또한, 제가 처음에 설계했던 UML과 최종본을 비교하면서 놓쳤던 부분을 확인할 수 있었고, 수업 내용을 응용하는 점에서 복습에도 도움이 되었습니다. 소규모의 팀 프로젝트 경험을 쌓아서 좋았고, 다들 정성을 다해 작업해주셔서 감사하게 생각합니다.

신창만: 이번 프로젝트를 진행하면서 자바에서 객체지향 설계와 클래스 간의 역할 분리에 대해 더욱 이해할 수 있었습니다. 프로젝트를 진행하면서 부족한 부분도 아쉬웠지만, 좋은 경험이 되었습니다. 함께 프로젝트를 진행해 주신 팀원분들께 감사드립니다.

오수빈: 첫 프로젝트를 진행하며 어떤 식으로 프로젝트가 진행되는지 전반적인 흐름을 느끼고 배울 수 있었습니다. 어떤 도구를 사용하는지, 어떻게 팀원간의 협업이 이루어 지는지 등을 몸소 느끼며 많은게 새롭고 흥미로웠습니다. 해매는 순간도 있었지만 팀원들의 도움의 손길 덕분에 많이 배우고 성장 할 수 있었습니다. 감사합니다.